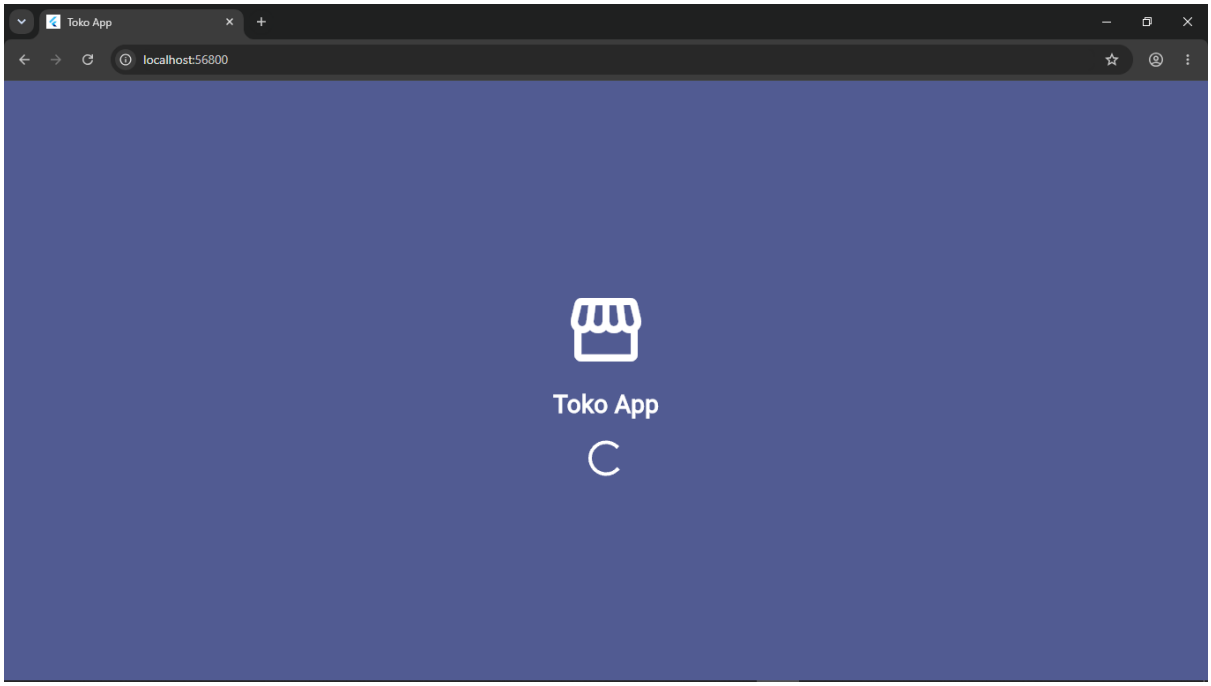
NAMA : MIFTAHUL JANNAH

NPM : 24011130444

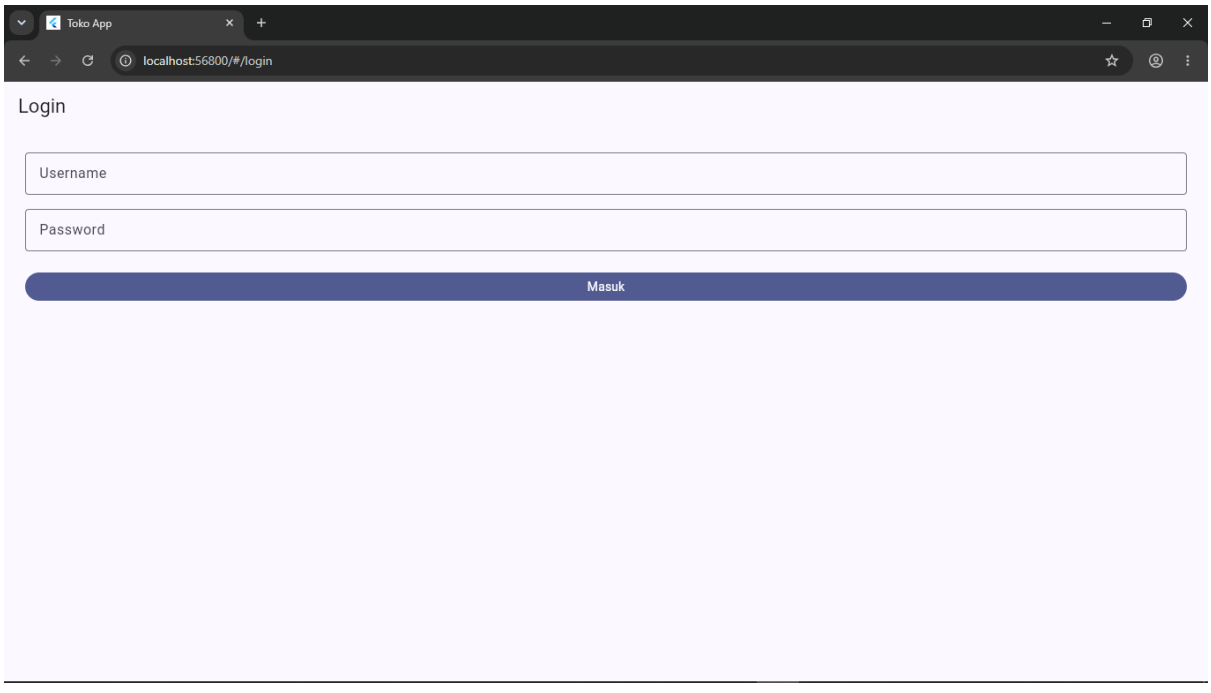
PRODI : D3 – MANAJEMEN INFORMATIKA

- BAGIAN A-

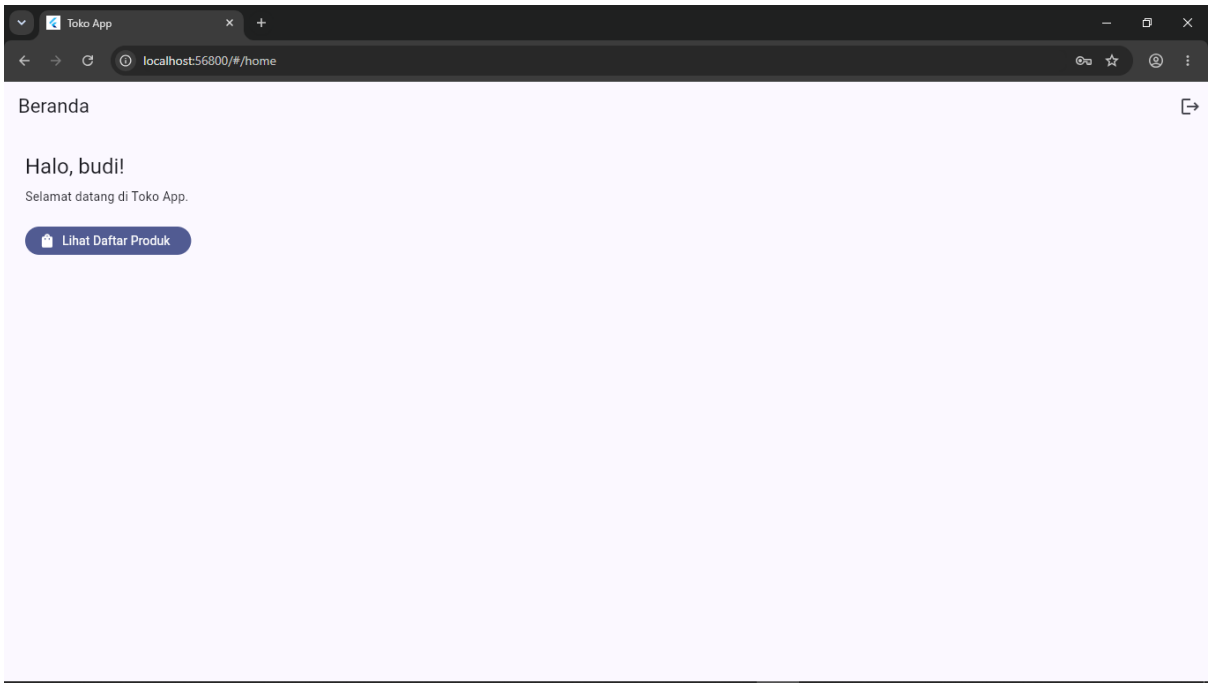
Tampilan splash



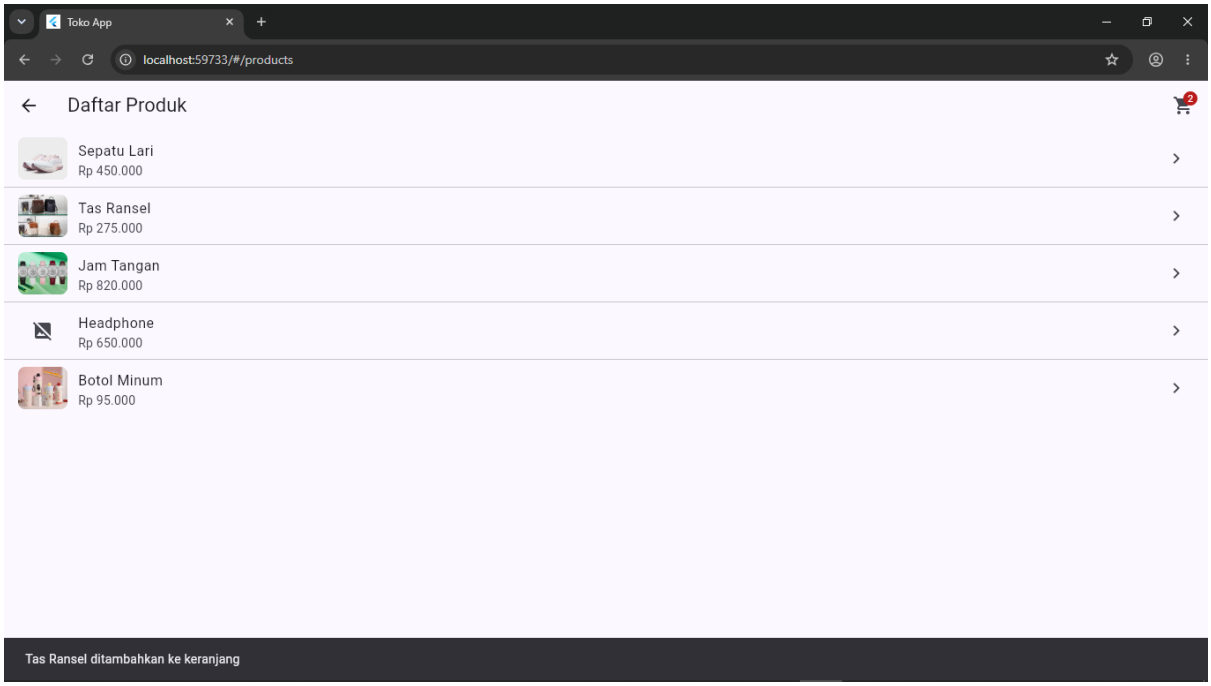
Tampilan login



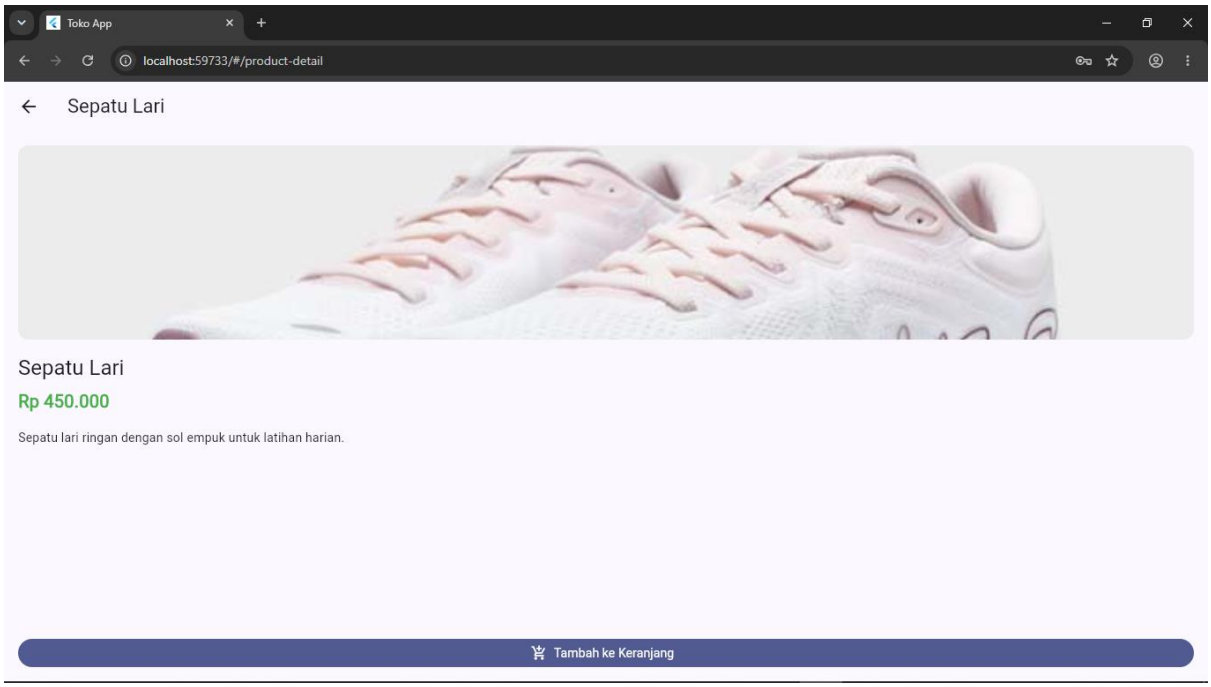
Membuka Beranda dan menampilkan "Halo, <username>!".



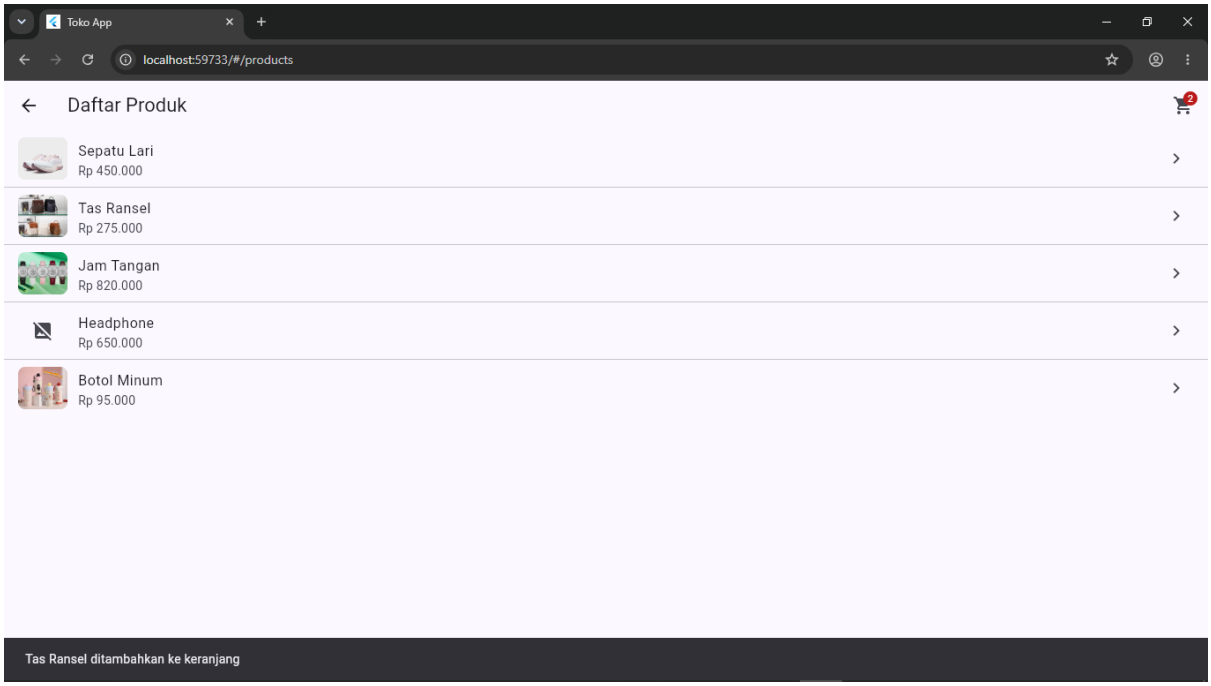
Daftar Produk menampilkan 5 produk lengkap dengan gambar dan harga.



Nama, harga, gambar, dan deskripsi produk tampil benar di halaman Detail.

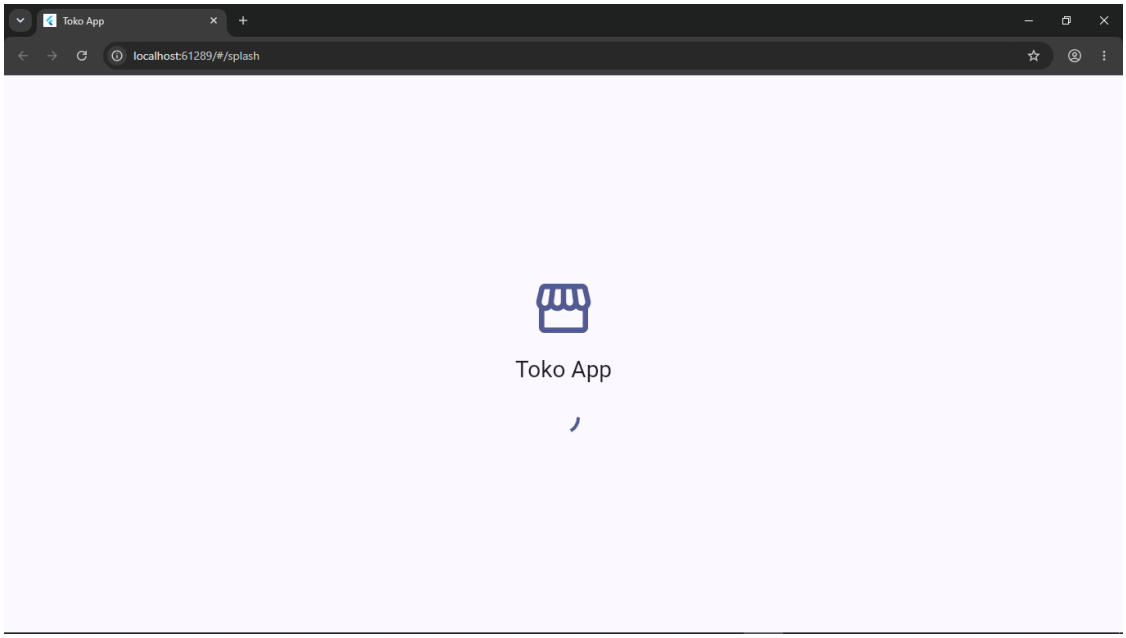


Tombol Tambah ke Keranjang kembali ke Daftar Produk dan memunculkan Snackbar; angka badge keranjang bertambah.

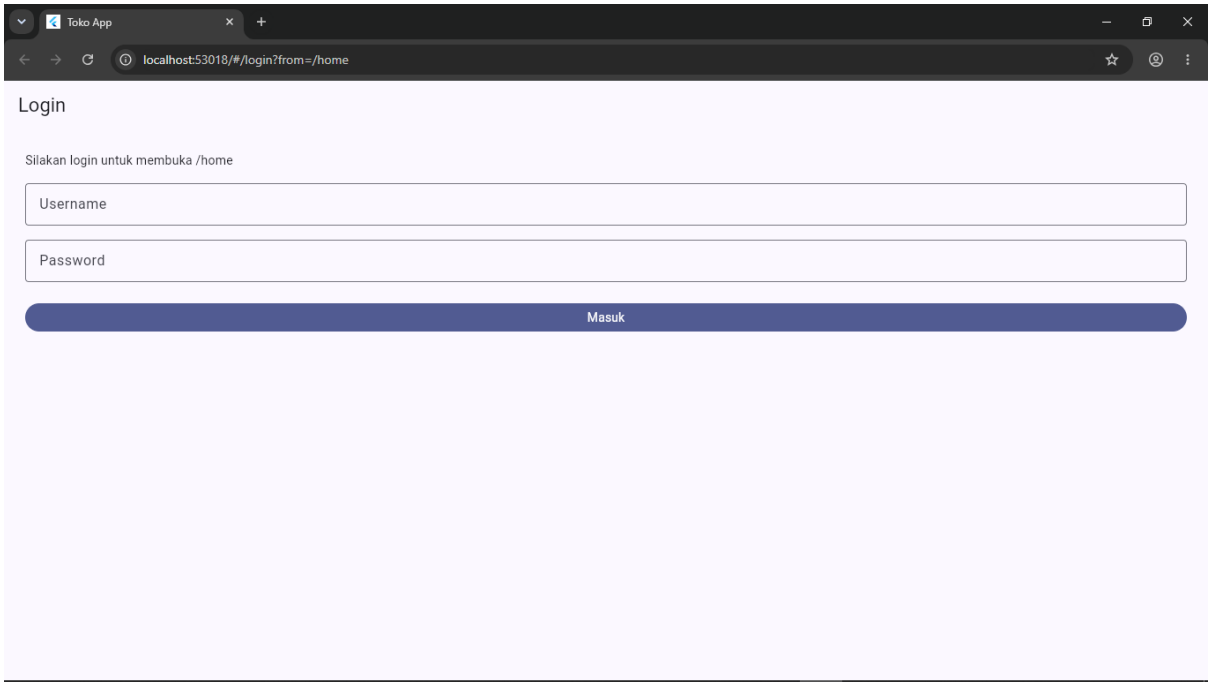


-BAGIAN B -

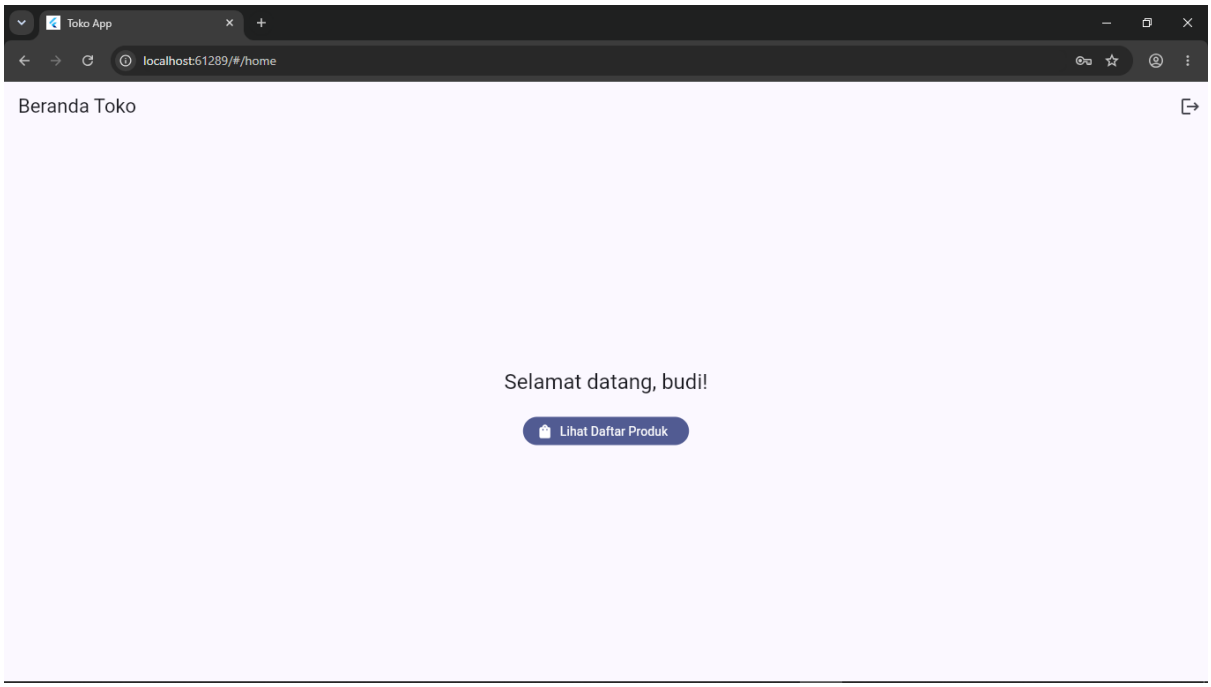
Tampilan splash



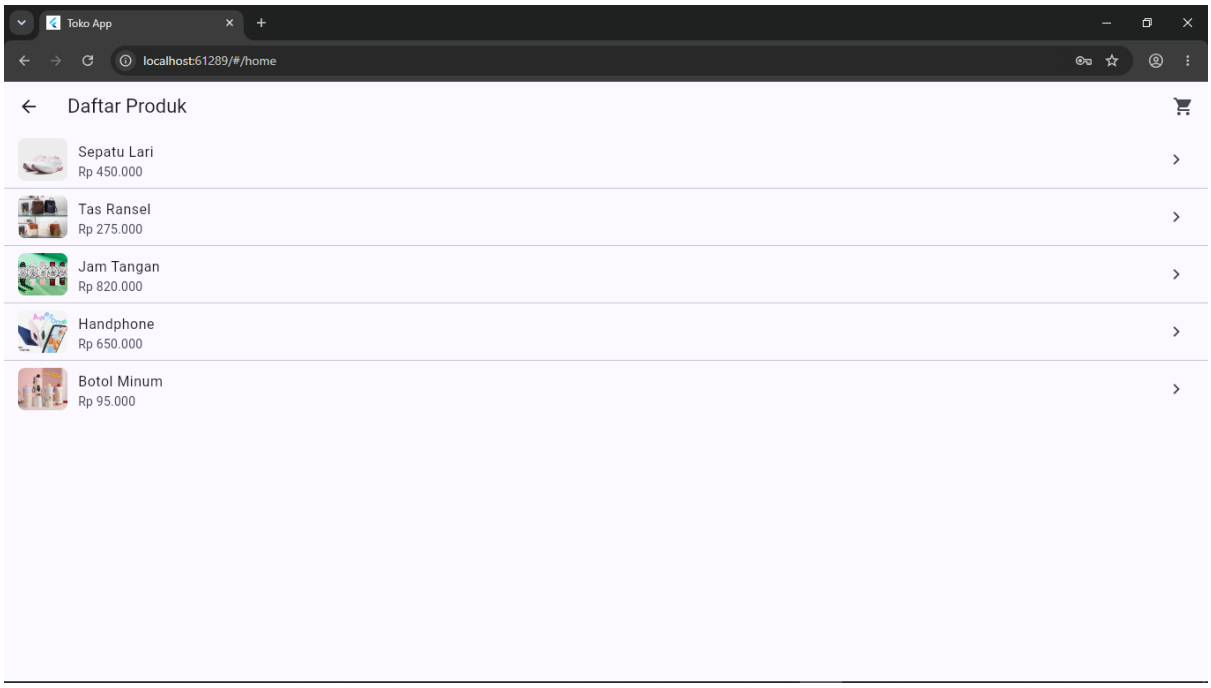
Aplikasi dibuka langsung menampilkan Login dengan tulisan "Silakan login untuk membuka /home".

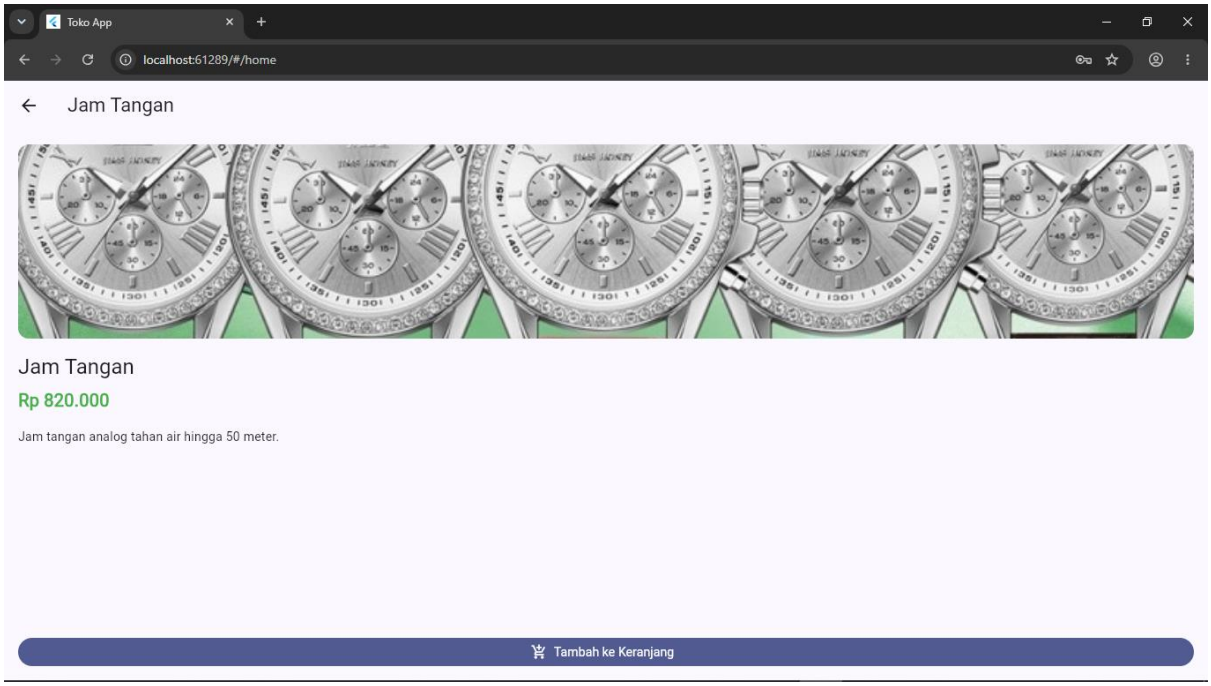


Setelah login, Beranda tampil dan menyapa nama pengguna.

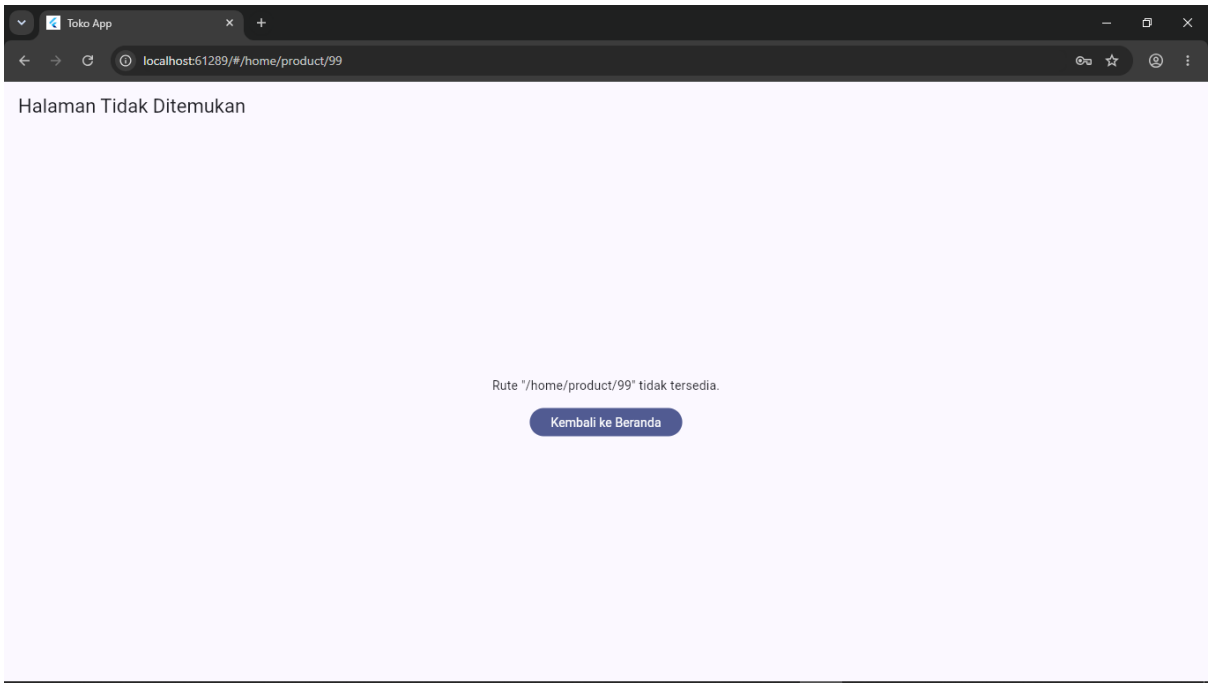


Mengetuk produk membuka detail; tombol back di AppBar kembali ke Beranda.

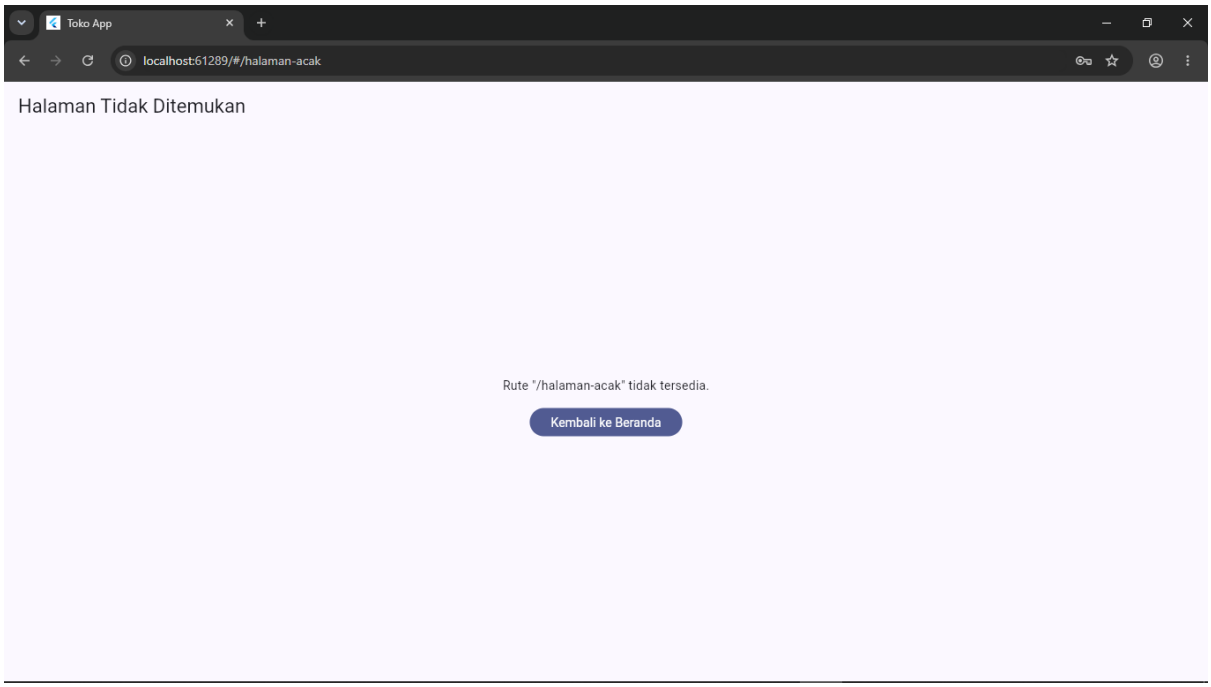




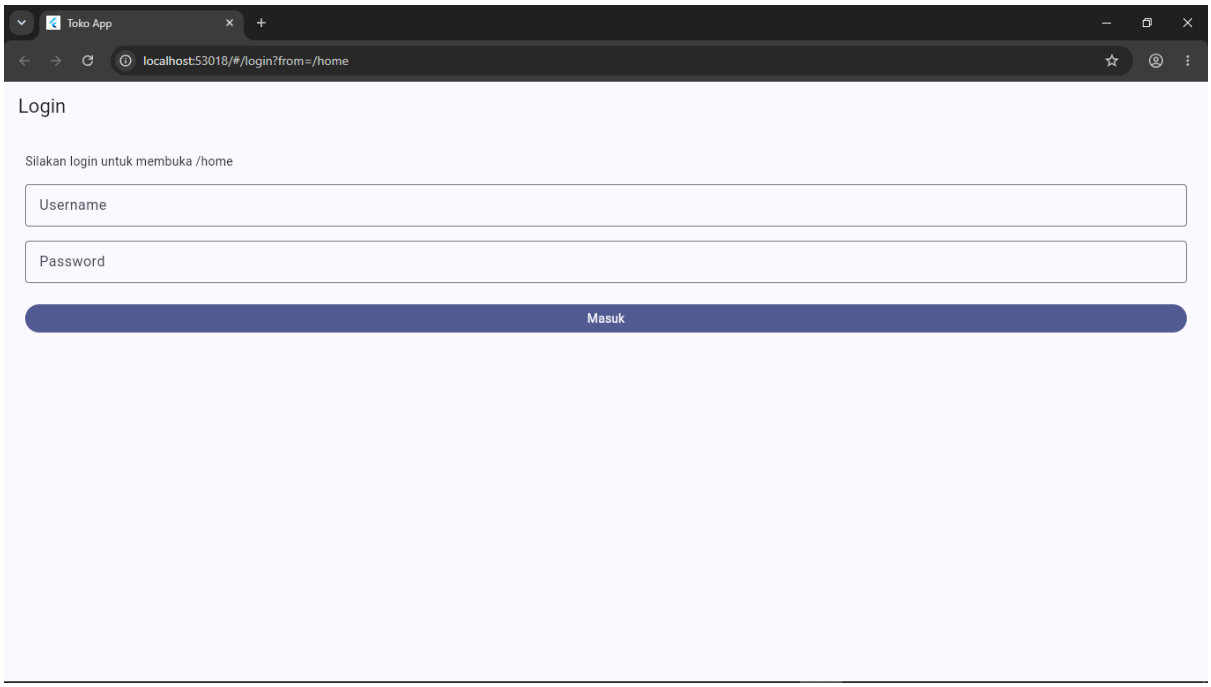
Di Chrome, ketik /#/home/product/99 saat sudah login → muncul "Produk tidak ditemukan".



Di Chrome, ketik /#/halaman-acak → muncul NotFoundPage.

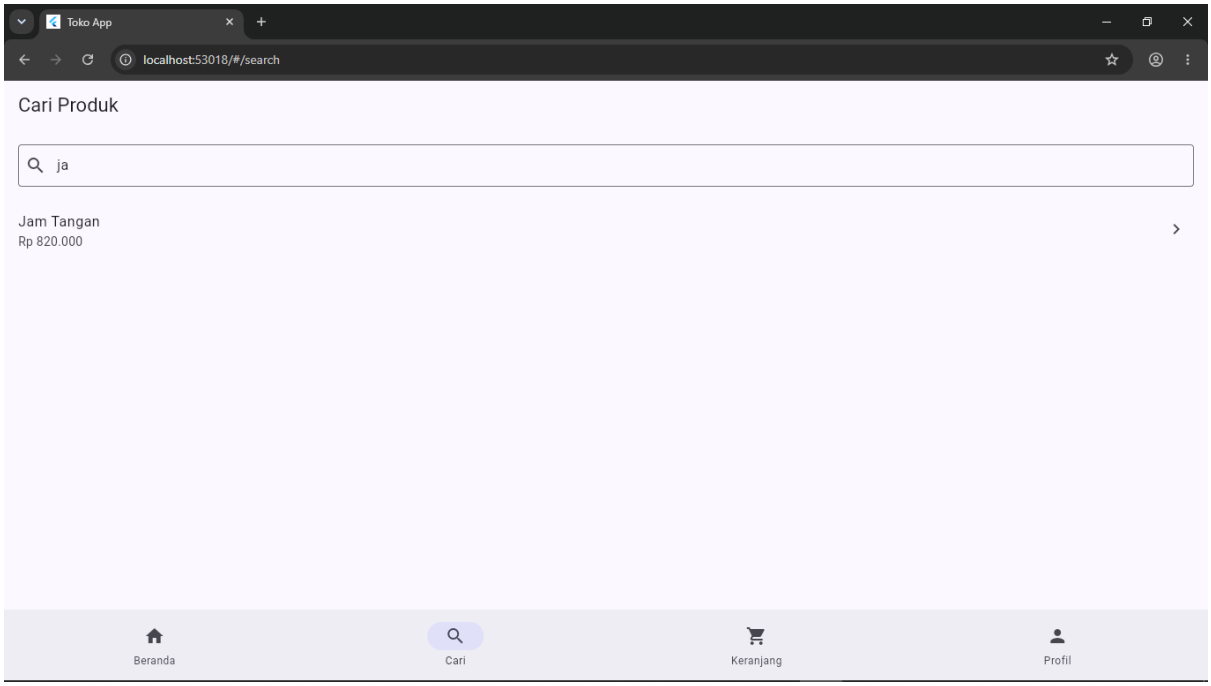


-BAGIAN C –

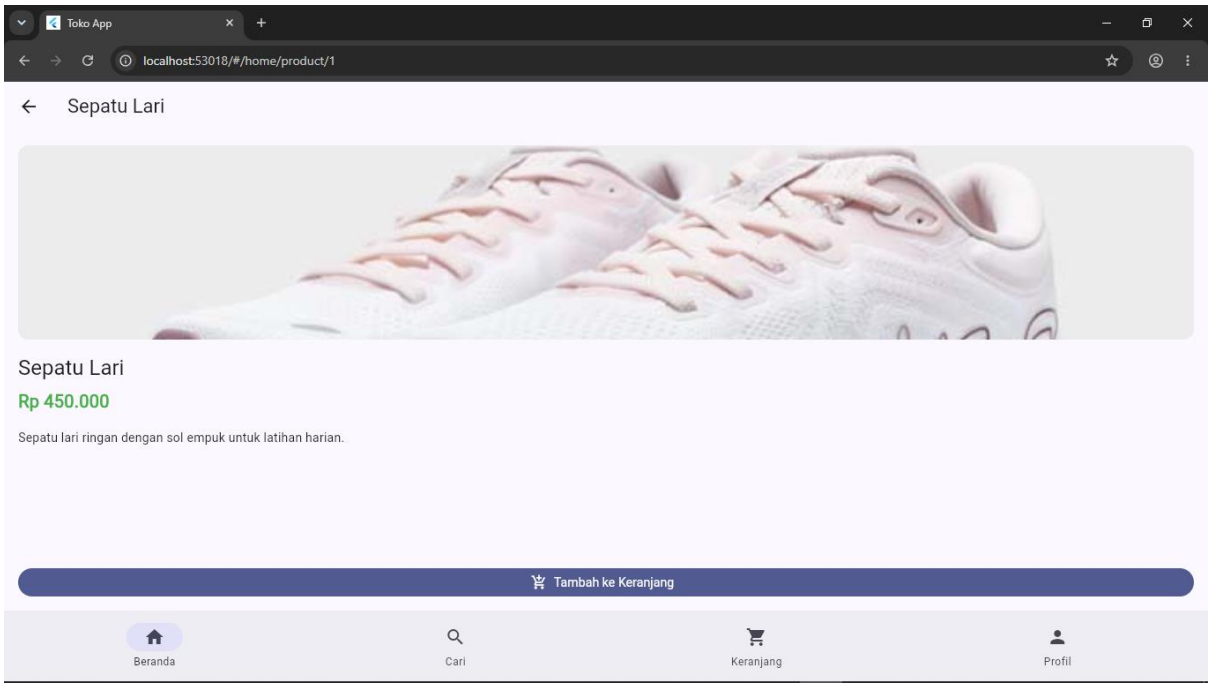

Toko App
localhost:53018/#/login?from=/home
Login
Silakan login untuk membuka /home
Username
Password
Masuk


Toko App
localhost:53018/#/home
Beranda
Halo, budi!
Produk pilihan untuk Anda:
Sepatu Lari
Rp 450.000
Tas Ransel
Rp 275.000
Jam Tangan
Rp 820.000
Headphone
Rp 650.000
Botol Minum
Rp 95.000
Beranda
Cari
Keranjang
Profil

Di tab Cari, ketik "ja", pindah ke tab Beranda, lalu kembali ke Cari: teks "ja" dan hasil Pencarian masih ada.



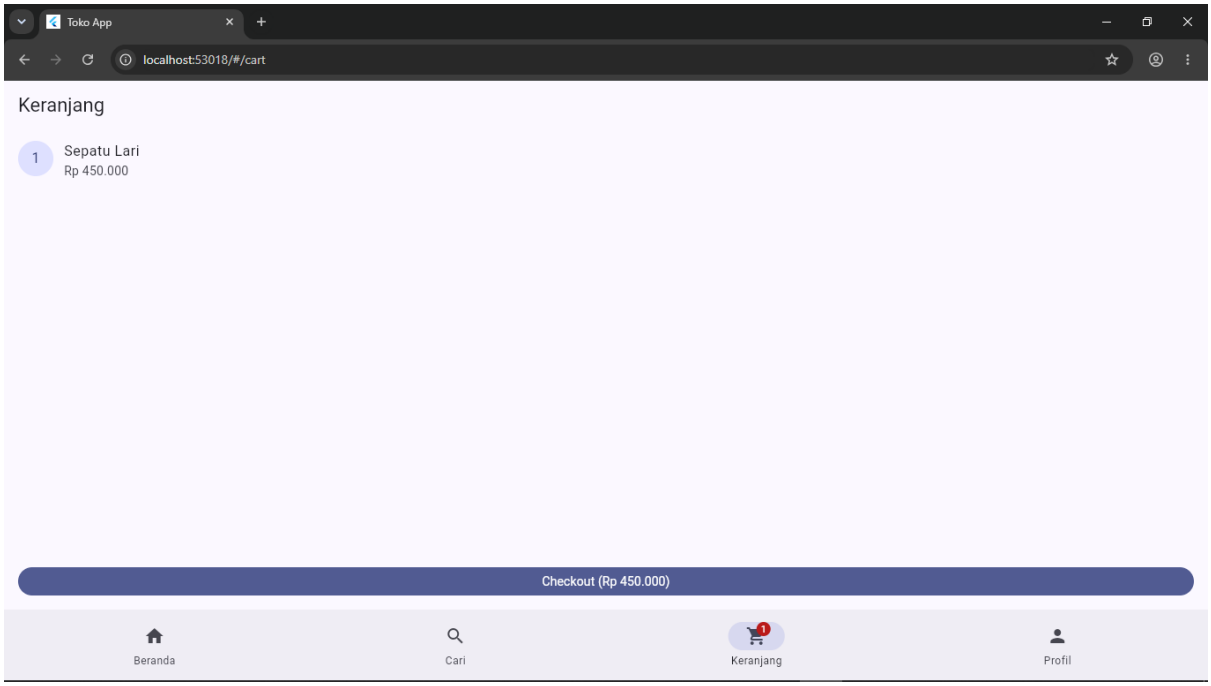
Buka detail produk di tab Beranda, pindah ke tab Profil, kembali ke Beranda: halaman detail masih terbuka.

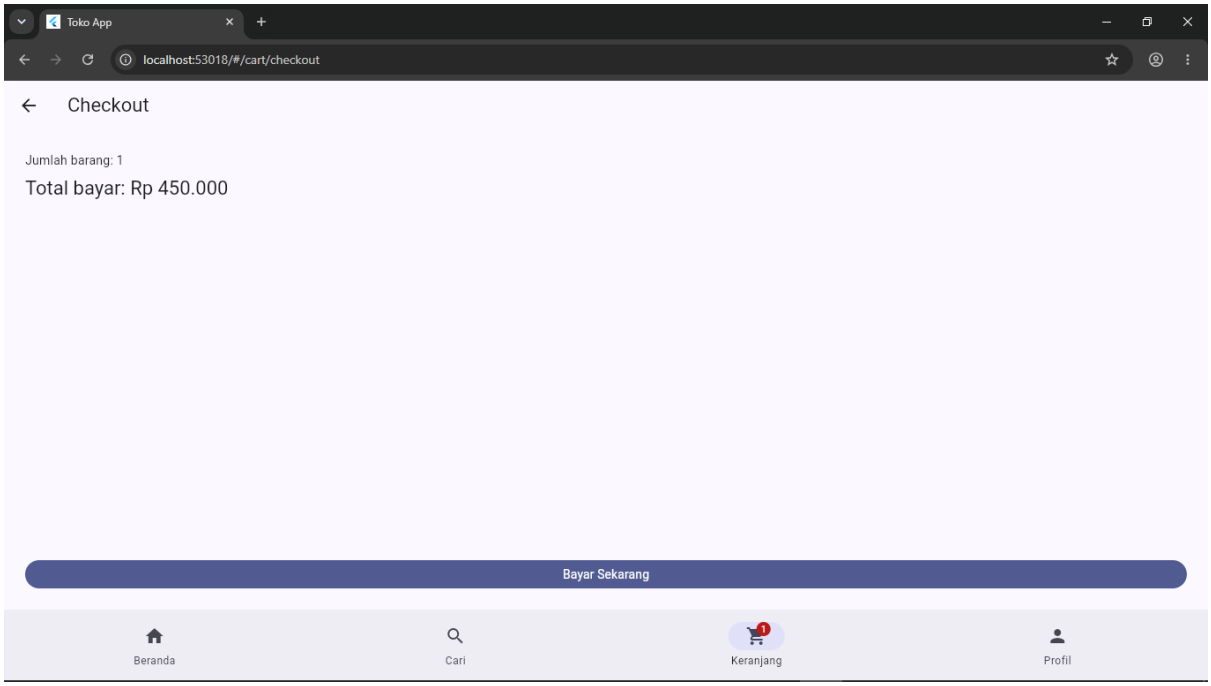


Tambah ke Keranjang menampilkan Snackbar dan angka badge pada ikon Keranjang bertambah.

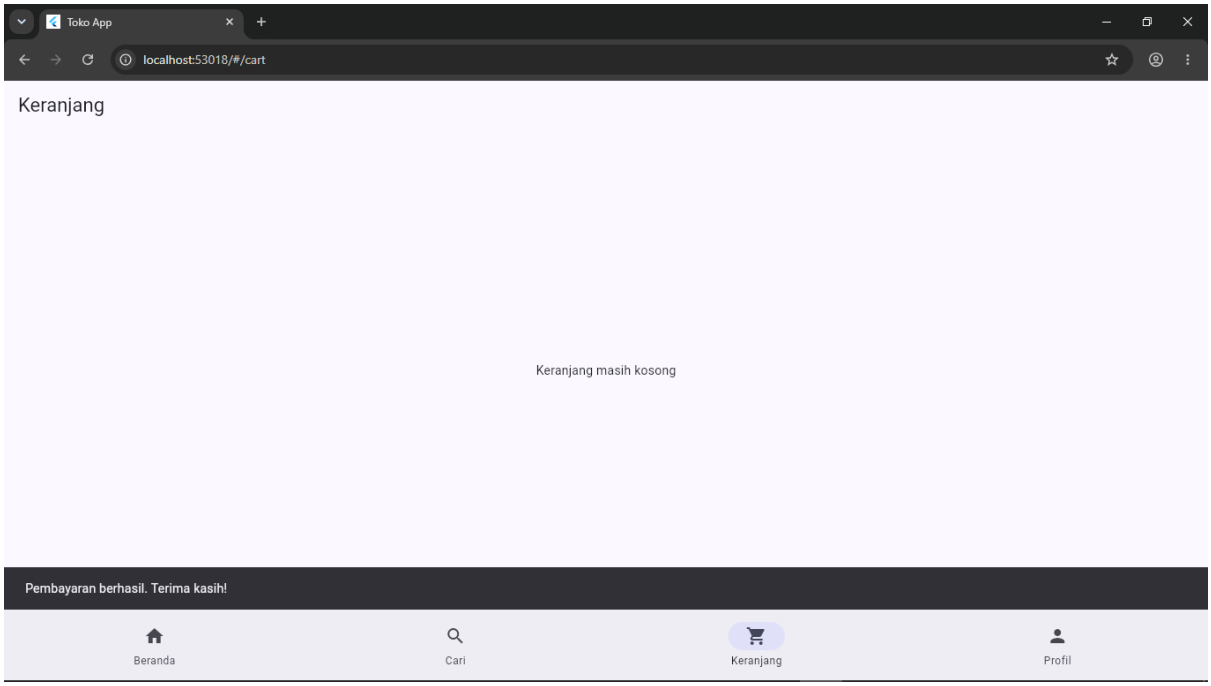


Keranjang → Checkout membuka /cart/checkout; NavigationBar tetap terlihat.

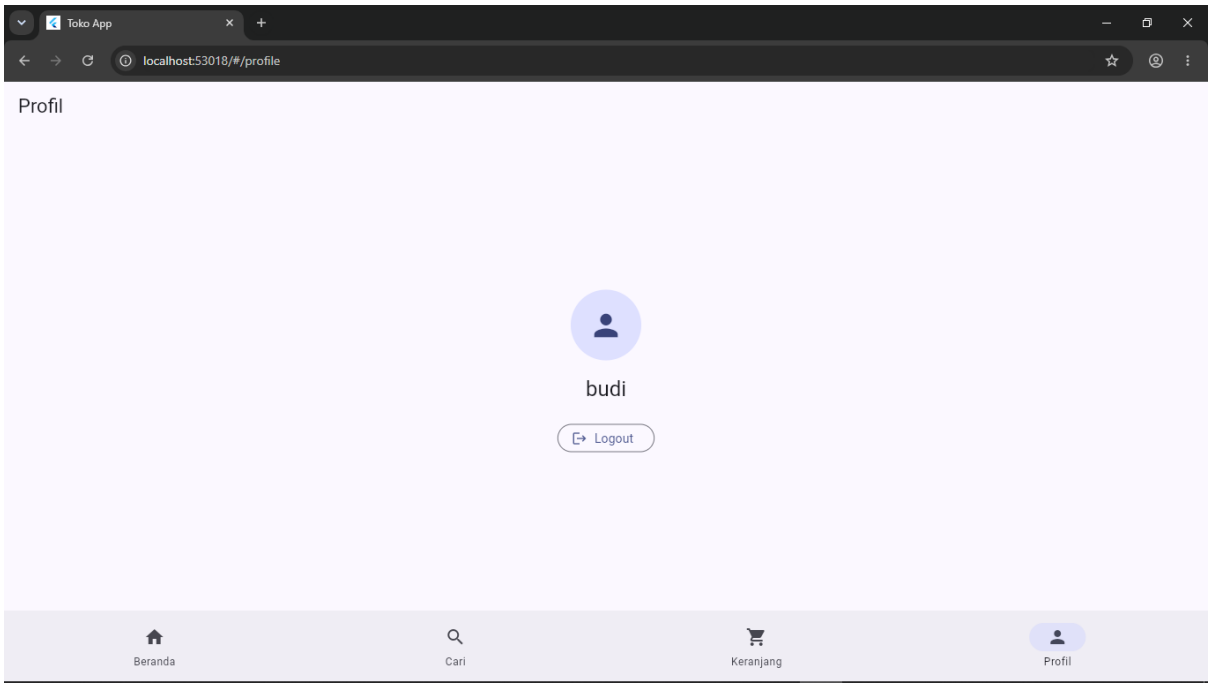




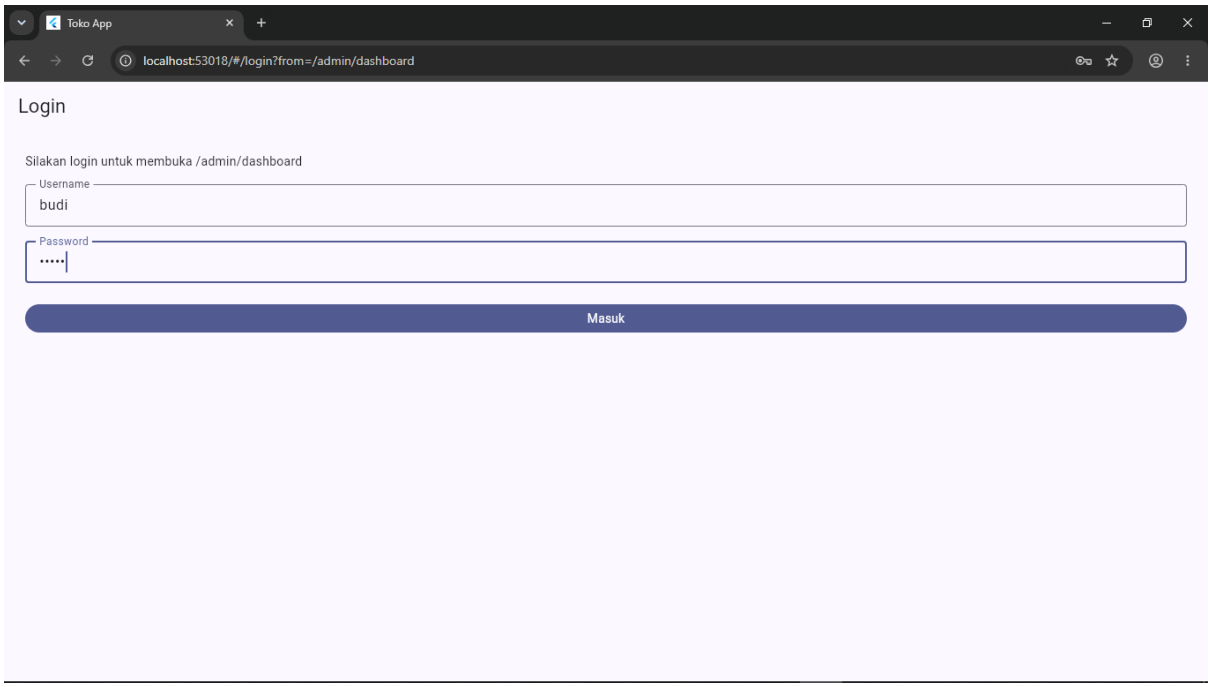
Bayar Sekarang mengosongkan keranjang dan badge hilang.



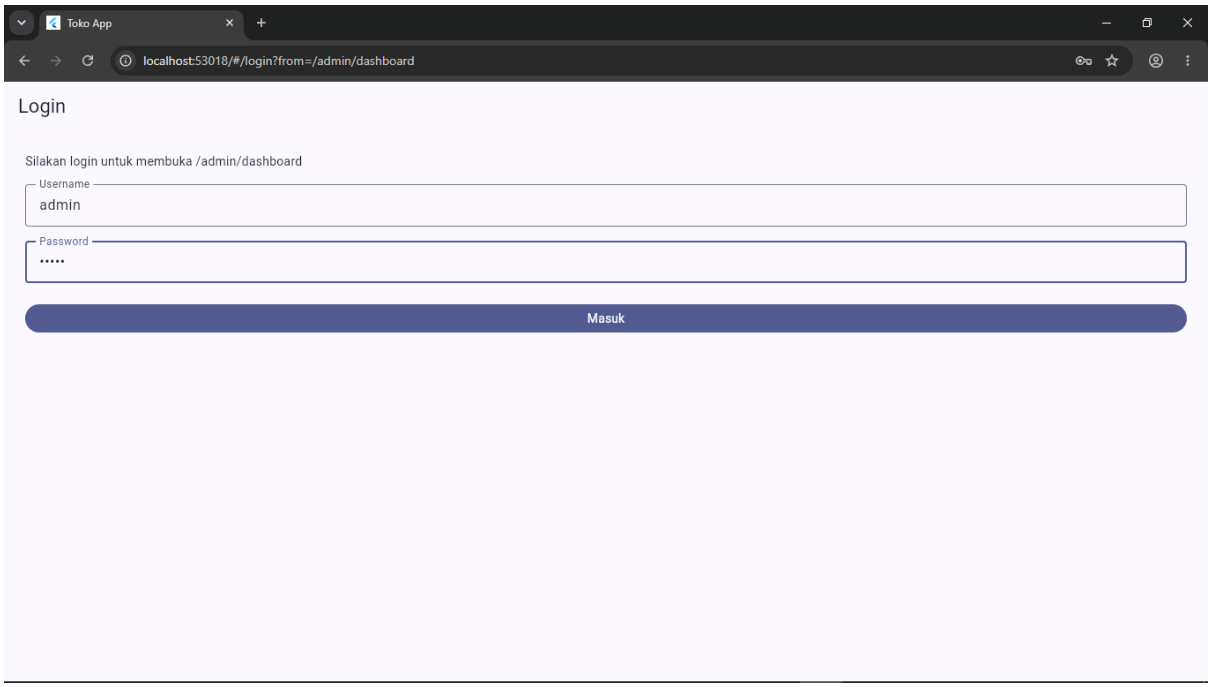
Logout dari tab Profil kembali ke Login; tombol back tidak membuka tab lagi.

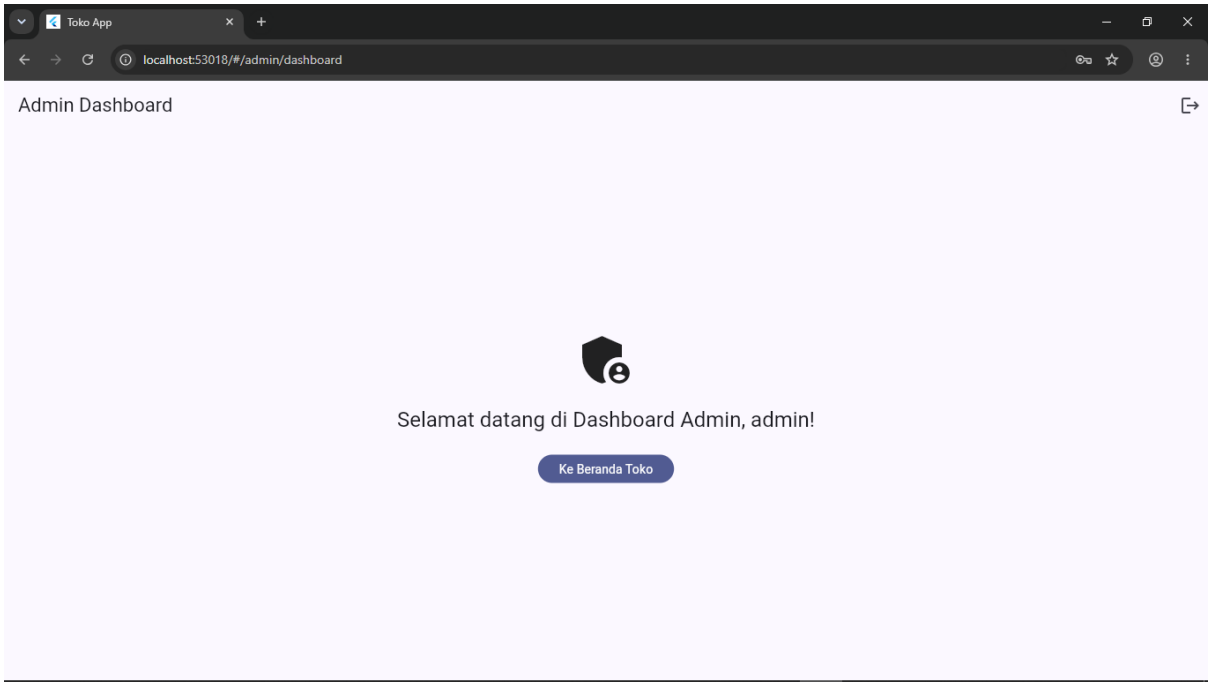


- TANTANGAN BONUS-



Sebagai admin




Toko App
localhost:53018/#/admin/dashboard
Admin Dashboard
Selamat datang di Dashboard Admin, admin!
Ke Beranda Toko

-PERTANYAAN ANALISIS –

1. (Bagian A) Mengapa SplashScreen berpindah ke Login menggunakan pushReplacementNamed(), bukan pushNamed()? Buktikan dengan mencoba keduanya.

   Jawab : Fungsi pushReplacementNamed() dipakai supaya halaman SplashScreen langsung diganti dan dihapus dari tumpukan (stack) navigasi saat pindah ke halaman Login. Kalau kita pakai pushNamed(), halaman splash cuma ditumpuk di bawah halaman Login. Berdasarkan hasil percobaan langsung, saat pakai pushNamed() dan tombol Back ditekan di halaman Login, aplikasi malah balik lagi ke layar splash yang sudah selesai. Sedangkan dengan pushReplacementNamed(), menekan tombol Back di halaman Login langsung menutup aplikasi karena halaman splash sudah tidak ada lagi di stack.
2. (Bagian A) Mengapa kita perlu memeriksa mounted setelah await Navigator.pushNamed(...)?

   Jawab : Karena await itu proses menunggu yang berjalan secara asinkron sampai pengguna selesai dari halaman tujuan. Selama waktu tunggu tersebut, ada kemungkinan halaman asal sudah ditutup atau dihancurkan (unmounted) dari tampilan layar. Pengecekan if (!mounted) return; berfungsi sebagai pengaman agar aplikasi tidak crash atau muncul error saat mencoba memanggil setState() atau memakai context pada halaman yang sudah tidak aktif lagi.
3. (Bagian A vs B) Pada Bagian A, halaman detail menerima objek Product lewat arguments; pada Bagian B menerima id dari URL. Jelaskan kelebihan cara kedua untuk deep link dan Flutter Web.

   Jawab : Di Flutter Web dan deep link, pengguna bisa langsung membuka halaman tertentu lewat tautan URL (misalnya #/home/product/3) atau menekan tombol refresh di browser tanpa harus lewat halaman daftar produk dulu. Kalau kita mengandalkan pengiriman objek Product lewat arguments di memori, datanya bakal kosong (null) saat link dibuka langsung. Sebaliknya, dengan menangkap id langsung dari URL, halaman detail bisa mencari dan memuat datanya sendiri kapan saja berdasarkan id tersebut.
4. (Bagian B) Apa perbedaan context.go('/home/product/3') dan context.push('/home/product/3')? Mengapa keduanya tetap menampilkan tombol back?

   Jawab : Perbedaannya ada di cara mengatur tumpukan halamannya. context.go() menyusun ulang susunan halaman mengikuti struktur rute di URL, sedangkan context.push() langsung menumpuk halaman baru di atas halaman yang sedang dibuka tanpa mengubah struktur dasarnya. Keduanya sama-sama tetap memunculkan tombol back karena rute product/:id didaftarkan sebagai rute anak (sub-route) di dalam /home, sehingga GoRouter otomatis menaruh /home di bawah /home/product/3.

5. (Bagian B) Jelaskan alur lengkap sejak pengguna yang belum login membuka /#/home/product/3 hingga detail produk tampil. Sebutkan peran redirect, from, dan refreshListenable.

   Jawab : Saat pengguna yang belum login membuka #/home/product/3, fungsi redirect langsung mencegatnya sebelum halaman dibangun karena mendeteksi status login masih false. GoRouter lalu menyimpan URL tujuan tadi ke dalam parameter from dan mengarahkan pengguna ke /login?from=%2Fhome%2Fproduct%2F3. Begitu pengguna mengisi form dan menekan tombol Masuk, authState.login() dijalankan dan memicu notifyListeners(). Karena authState terhubung dengan refreshListenable, GoRouter otomatis mengecek ulang fungsi redirect. Kali ini pengguna sudah tercatat login, sehingga GoRouter membaca alamat di parameter from dan langsung mengantar pengguna ke halaman /home/product/3.

6. (Bagian C) Apa yang terjadi pada tab Cari jika StatefulShellRoute.indexedStack diganti dengan ShellRoute biasa? Jelaskan mengapa.

   Jawab : Kalau diganti memakai ShellRoute biasa, setiap kali kita pindah ke tab lain (misalnya dari Cari ke Beranda), halaman di tab Cari bakal dihancurkan (disposed) dan dibuat ulang dari nol saat kita kembali lagi ke sana. Akibatnya, kata kunci pencarian yang sudah diketik dan hasil pencariannya bakal langsung hilang atau ter-reset setiap kali ganti tab.

7. (Soal Evaluasi no. 8) Seorang developer memeriksa status login di initState() setiap halaman lalu memanggil Navigator.pushReplacement ke Login. Sebutkan minimal tiga masalah pendekatan tersebut dan jelaskan mengapa redirect lebih baik.

   Jawab :

   - Kode jadi berulang-ulang: Kita harus mengetik pengecekan status login yang sama di setiap halaman rahasia, sehingga kode jadi berantakan dan susah dirawat kalau ada perubahan aturan.
   - Halaman sempat terlihat sekilas (flicker): Karena initState() berjalan saat halaman mulai dibangun, tampilan halaman yang seharusnya terkunci bisa sempat muncul sepersekian detik sebelum pindah ke Login
   - Susah mengatur deep link dan halaman tujuan awal: Menyimpan alamat halaman yang tadinya mau dibuka pengguna lalu mengembalikannya setelah login jauh lebih ribet dan rawan bug kalau diurus manual di tiap halaman, dibanding dipusatkan di satu pintu lewat redirect milik GoRouter yang otomatis menjaga seluruh rute sebelum halaman sempat dirender.